

STARVIVORS — Master Game Design Document & AI Development Plan

Document Status

Current source of truth for STARVIVORS.

The older GDD is now reference-only. This document removes the ForgePlay Engine assumption and rebuilds the game direction around a modular HTML5/TypeScript project intended for eventual Steam release.

MASTER GDD

1. Game Identity

Working Title

STARVIVORS

Genre

Top-down space survival roguelite with physics-influenced movement, black hole hazards, and escalating enemy pressure.

Core Inspiration

- Vampire Survivors-style survival pacing, upgrade pressure, and post-run shop progression
- Asteroids-style space movement, momentum, drifting, and wrapping world behavior
- Arcade roguelite build variety
- Simple geometric visuals with strong particles and readable UI

Core Hook

Pilot a ship through a massive wrapping space arena where enemies, debris, projectiles, pickups, and the player are all affected by moving black holes. Destroy ships, break apart their wreckage, gather scrap, bank upgrades, and survive long enough to escape the collapsing battlefield.

Design Pillars

1. Gameplay and loop first, graphics second.
2. Simple shapes, strong readability, and satisfying particles.

3. Movement should feel floaty, physical, and skill-based.
 4. Black holes are a central gameplay mechanic, not just background hazards.
 5. Upgrades should create build momentum during a run.
 6. Permanent progression should be simple and shop-based.
 7. The codebase must be modular so ChatGPT and Codex can work on isolated systems safely.
 8. WIP content may be visible, but only implemented content should affect active gameplay.
-

2. Target Platform and Technical Direction

Primary Build Target

HTML5 game packaged for desktop release on Steam.

Development Target

Browser-playable prototype first.

Steam Target

Desktop executable wrapper after the browser game loop is fun and stable.

Recommended Stack

- Language: TypeScript
- Game framework: Phaser 3 or Pixijs
- Build tool: Vite
- Desktop wrapper: Electron or Tauri
- Repository: GitHub
- AI coding: Codex
- Planning, design, review, prompt writing: ChatGPT

Technical Principle

Do not build the game as one giant HTML file. Use modular files from the beginning. The project should be structured so Codex can safely work on one system at a time.

3. Core Game Concept

The player pilots a small spacecraft in a huge wrapping space arena. Enemies spawn continuously and escalate over time. Destroyed enemies break into hazardous drifting debris. Debris can be destroyed for XP and may drop collectible scrap or minerals.

The player gains XP directly from destroying ships and debris. There are no XP gems. When the XP bar fills, the player earns a banked upgrade. Banked upgrades are not forced immediately; the player chooses when to open the upgrade screen. Gameplay pauses only when the player chooses to spend an upgrade.

Black holes drift through the arena, pulling matter into orbit and eventually consuming it. They can destroy enemies and debris, but they can also consume pickups, curve projectiles, and kill the player.

4. Round Structure

Round Length

MVP target: 15 minutes.

Future mode: 30 minutes.

Win Condition

Survive until the timer reaches zero.

Loss Condition

The player loses when hull reaches zero or the player is consumed by a black hole.

Run Flow

1. Select ship.
 2. Start run.
 3. Move through the arena using floaty thrust-based movement.
 4. Destroy enemies and hazardous debris.
 5. Gain XP directly as number ticks.
 6. Fill the XP bar and bank upgrades.
 7. Press the upgrade button when safe.
 8. Choose 1 of 3 in-run weapon upgrades.
 9. Collect scrap/mineral pickups for end-run currency.
 10. Avoid or exploit moving black holes.
 11. Survive until the timer reaches zero or die.
 12. View results and resource tally.
 13. Spend currency in the main-menu shop.
 14. Start another run.
-

5. Arena

Arena Size

The full arena is 9 by 9 of the main viewport.

Definition:

```
Arena width = viewport width × 9  
Arena height = viewport height × 9
```

Example:

```
If the viewport is 1920×1080,  
the arena is 17,280×9,720 world units.
```

Camera

The camera follows the player and displays one viewport-sized section of the arena at a time.

Boundary Behavior

The world wraps at the outer arena edges.

Wrapping applies to:

- Player
- Enemies
- Enemy debris
- Projectiles, depending on weapon rules
- Pickups, if enabled
- Black holes, unless a specific mode uses edge bounce

Arena Design Goal

The arena should feel like open space, not a small box. The 9×9 viewport size gives the player room to drift, kite, gather enemies, avoid black holes, and recover from danger.

6. Player Movement

Movement Feel

The player ship moves like an object in space:

- Acceleration-based movement
- Momentum and drifting
- Low friction
- Thrust control
- Braking or dampening
- Skill-based recovery

Controls

Keyboard and mouse default:

- W / Up: thrust
- A / D or Left / Right: rotate
- S / Down: brake or reverse thrust
- Mouse: aim weapons
- Left click: fire weapon slot 1
- Right click: fire weapon slot 2
- Space: dash / emergency burst
- Upgrade key/button: open banked upgrade selection when available

Controller support is planned but not required for the first playable prototype.

Movement Design Note

The player should feel endangered by momentum and gravity, but not helpless. Bad positioning should be punishable; good thrust control should let skilled players escape.

7. Black Hole System

Role

Black holes are the signature environmental hazard and tactical system.

MVP Behavior

- At least one black hole exists during the run.
- Black holes slowly move across the arena/world.
- Black holes affect all major physical entities.
- Black holes consume matter that reaches the center.
- Black holes have clear visual danger zones.

Affected Entities

Black holes pull:

- Player
- Enemies
- Enemy debris
- Asteroids
- Projectiles
- Scrap/mineral pickups, unless exempted for balance
- Particles when visually useful

Gameplay Functions

Black holes should:

- Create positional danger
- Bend projectile paths
- Break up enemy formations
- Consume debris
- Destroy enemies
- Threaten valuable pickups
- Force movement decisions
- Reward tactical positioning

Visual Behavior

Black holes should use simple shapes and effects:

- Dark core
- Glowing edge
- Accretion ring
- Orbiting debris particles
- Starfield bending or flowing inward
- Gravity radius indicator
- Danger radius indicator
- Consumption particles

Movement Behavior

Default direction: slow drifting movement across the full arena.

Recommended MVP behavior: black holes wrap around arena edges.

Design Constraint

The player should always understand why the black hole is dangerous. Death should feel avoidable, not arbitrary.

8. Ships

Ship Philosophy

Each ship should meaningfully change how the player moves, survives, or attacks.

MVP Ships

The MVP includes two playable ships to test two distinct playstyles:

1. Interceptor — ranged shooter
2. Bulwark — melee ramming ship

Additional ships may be visible with WIP tags but are not playable until implemented.

Interceptor

Role: ranged shooter.

Starting weapon: Pulse Cannon.

Playstyle:

- Kite enemies
- Aim and fire from range
- Clear debris safely
- Avoid being cornered by black holes

Strengths:

- Flexible
- Safe compared to melee
- Easy to understand
- Good baseline for testing ranged combat

Weaknesses:

- Fragile
 - Low starting fire rate
 - Needs upgrades to handle crowds well
-

Bulwark

Role: melee impact ship.

Starting weapon: Ramming Shield.

Playstyle:

- Gather enemies
- Charge through groups
- Deal heavy impact damage
- Back off when shield is low
- Wait for shield recharge before re-engaging

Strengths:

- High burst group damage
- Durable when shielded
- Strong against clustered enemies
- Unique playstyle

Weaknesses:

- Dangerous when shield is depleted
 - Requires close contact
 - More vulnerable to black hole positioning mistakes
 - Needs space to retreat and recharge
-

9. Weapons

Weapon Philosophy

Weapons should be easy to understand, visually distinct, and upgradeable. The player should manage only a small number of active weapons during a run.

Weapon Slots

The player has 2 active weapon slots per run.

Slot 1:

- Ship-specific starting weapon
- Defines ship identity

Slot 2:

- Acquired during the run
- Creates build variety

MVP Starting Weapons

- Interceptor: Pulse Cannon
 - Bulwark: Ramming Shield
-

Pulse Cannon

Weapon type: primary ranged weapon.

Starting ship: Interceptor.

Input: left click or hold fire.

Behavior:

The Pulse Cannon fires bright energy pulses in the aimed direction. It is the baseline ranged weapon and should be reliable, readable, and highly upgradeable.

Visual:

- Small glowing circular or oval projectile
- Short trail
- Clean impact burst

Role:

- Medium-range damage
- Safe enemy clearing
- Debris cleanup
- Baseline weapon for tuning enemy health and upgrade pacing

MVP base stats:

```
Damage: medium-high per shot
Fire cooldown: 1.25 seconds
Projectile count: 1
Projectile speed: medium-fast
Projectile size: medium
Pierce: 0
Range: medium-long
Reload: none for MVP
Black hole influence: medium
```

Design note:

The Pulse Cannon starts deliberately slow. A slower starting fire rate creates harder early play and gives progression more room to matter.

MVP in-run upgrades:

- Damage +
- Fire Rate +
- Projectile Count +
- Projectile Speed +
- Pierce +
- Projectile Size +

Future/WIP upgrades:

- Pulse Splitter
- Pulse Overcharge
- Pulse Stability
- Chain Pulse
- Ricochet Pulse
- Explosive Pulse

Ramming Shield

Weapon type: primary melee/impact weapon.

Starting ship: Bulwark.

Input: movement-based, with optional active charge or dash synergy.

Behavior:

The Bulwark has a rechargeable ramming shield. When the shield has charge, collisions with enemies and debris deal impact damage. The player uses momentum to ram through enemies, then backs away once shield charge nears zero.

Core loop:

1. Shield charges over time.
2. Player gathers enemies.
3. Player accelerates into enemies or debris.
4. Ram deals damage based on speed and shield charge.
5. Shield drains from impacts.
6. Player disengages and waits for recharge.
7. Repeat.

MVP behavior:

- Damage scales with current speed.
- Damage scales with shield charge.
- Shield absorbs collision damage while active.
- Shield loses charge on impact.
- If shield is empty, collision damages hull.
- Shield begins recharging after a short delay without impacts.

MVP in-run upgrades:

- Ram Damage +
- Ram Shield Capacity +
- Ram Shield Recharge +
- Impact Radius +
- Shield Loss Reduction +
- Dash Ram Bonus +

Future/WIP upgrades:

- Shockwave Ram
- Reflective Shield
- Black-Hole Resistant Shield
- Piercing Charge
- Shield Explosion on Break

10. Weapon Attachments

Role

Weapon attachments are potential loot drops that modify weapon behavior during a run.

MVP Status

Attachments may be visible as WIP content but do not need to be fully wired for the first playable MVP.

Example Attachments

- Split shot
- Chain lightning
- Burning plasma
- Homing module
- Ricochet module
- Gravity-resistant rounds
- Explosive payload
- Shield-piercing rounds

- Increased projectile mass
- Black-hole-reactive rounds
- Ram shockwave module
- Ram shield capacitor

Design Note

Attachments should be data-driven and tagged by compatible weapon type.

11. Enemies, Asteroids, and Hazard Debris

Enemy Philosophy

Enemies should be readable shapes with distinct movement and attack behavior.

Enemy Categories

- Chasers
- Shooters
- Bombers
- Tanks
- Swarmers
- Splitters
- Shielded enemies
- Black-hole-resistant enemies
- Elites
- Bosses

Asteroids

Asteroids are natural drifting hazards.

They can:

- Drift through the arena
- Damage the player on contact
- Be destroyed
- Grant XP when destroyed
- Drop scrap/minerals
- Be pulled into black holes

Hazard Debris

Destroyed ships can break into drifting debris chunks.

Hazard debris is not a pickup. It is a physical object that remains in the world.

Hazard debris can:

- Drift with inherited momentum
- Damage the player on contact
- Have health
- Be destroyed
- Grant XP when destroyed
- Drop scrap/minerals
- Be pulled into black holes
- Be consumed by black holes

Enemy Breakup Behavior

Enemy destruction may spawn debris chunks.

Recommended MVP values:

Small enemy: 0-1 debris chunk
Medium enemy: 1-2 debris chunks
Large enemy: 2-4 debris chunks
Elite/Boss: future tuning

Design Constraint

Debris should add tactical clutter and cleanup decisions without making the arena unreadable.

12. XP and Upgrade System

XP Source

XP is awarded directly when the player destroys enemies, asteroids, or hazard debris.

There are no XP gems/fragments.

XP Feedback

XP appears as number ticks, bar movement, or subtle UI feedback similar to MMO-style resource gain.

XP Bar

The XP bar appears at the top of the screen, similar to Vampire Survivors.

When the bar fills:

- The player gains one banked upgrade.
- The XP bar rolls over toward the next level.
- Gameplay does not auto-pause.

Banked Upgrades

Upgrades are bankable infinitely.

There is no cap.

The UI should clearly display:

```
UPGRADE AVAILABLE x1  
UPGRADE AVAILABLE x2  
UPGRADE AVAILABLE x5
```

Upgrade Activation

The player chooses when to open the upgrade screen.

When opened:

- Gameplay pauses.
- One banked upgrade is spent after the player chooses an upgrade.
- The player chooses 1 of 3 upgrade cards.
- If additional upgrades remain banked, the player may choose another or return to gameplay.

In-Run Upgrade Rule

In-run upgrades are weapon-focused.

They may affect:

- Active weapon damage
- Attack speed
- Projectile count
- Projectile size
- Pierce
- Range
- Ram damage
- Ram shield behavior
- Weapon attachment effects

In-run upgrades should not be the main source of permanent ship/survival upgrades.

13. Pickups and End-Run Currency

Pickup Types

- Scrap
- Minerals
- Rare minerals, future
- Weapon upgrade drops, future/WIP
- Attachment drops, future/WIP
- Health/shield restore, optional

Important Distinction

Hazard debris and pickup debris are different.

Hazard debris:

- Physical object
- Has collision
- Has health
- Can hurt player

Scrap/mineral pickups:

- Collectible resource
- Used for end-run tally
- Converted into spendable currency

Currency Flow

During run:

- Player collects scrap/minerals.
- Values are tracked in the HUD.

After run:

- Scrap/minerals are tallied.
- Tally converts into credits/gold.
- Credits/gold can be spent in the shop.

MVP Economy Recommendation

Use one currency for MVP: Credits.

Scrap collected during a run converts into Credits after the run.

14. Shop and Meta-Progression

Shop Philosophy

The shop should be simple and direct, similar to Vampire Survivors.

There is no sphere grid.

Shop Location

The shop is accessible from the main menu and after the results screen.

Shop Currency

Credits earned from run resources.

Shop Categories

Ship/survival upgrades:

- Max Hull
- Shield Capacity
- Shield Recharge
- Thrust
- Turn Speed
- Brake Power
- Dash Cooldown
- Pickup Range

Weapon upgrades:

- Global Weapon Damage
- Attack Speed
- Projectile Size
- Projectile Count, carefully capped
- Reload Speed, future
- Starting Melee Range
- Melee Damage
- Ram Shield Capacity
- Ram Shield Recharge
- Attachment Drop Chance

Economy upgrades:

- Scrap Value
- Mineral Value
- Bonus Credits

- Shop Discount, future

Unlocks:

- New ships
- New weapon attachments
- New weapon types

MVP Shop Items

- Max Hull
- Shield Capacity
- Thrust
- Pickup Range
- Weapon Damage
- Attack Speed
- Melee Damage
- Ram Shield Recharge
- Scrap Value

Design Constraint

Shop upgrades should improve long-term progression but not remove the need for skilled movement and positioning.

15. UI and Screens

Required MVP Screens

- Main Menu
- Ship Select
- Shop
- Game HUD
- Upgrade Overlay
- Pause Menu
- Results Screen

Main Menu

Buttons:

- Start Run
- Ships
- Shop
- Settings
- Quit, desktop build only

Ship Select

Shows all planned ships visually.

Only implemented ships are selectable.

WIP ships should be visible but tagged.

Shop

Simple upgrade list/grid.

Each upgrade shows:

- Name
- Current level
- Effect
- Cost
- Purchase button
- WIP/Locked/Implemented status if needed

Game HUD

Required HUD elements:

- Timer
- Health/hull
- Shield
- XP bar
- Banked upgrade count
- Upgrade available button/prompt
- Current scrap/minerals
- Weapon slot display
- Dash charges
- Black hole warning indicators

Optional later:

- Minimap
- Enemy radar
- Boss health bar
- Attachment display

Upgrade Overlay

- Opens only when player chooses to spend a banked upgrade.
- Gameplay pauses.
- Shows 3 cards.

- Only implemented and gameplay-enabled upgrades should appear.
 - WIP upgrades may exist in data and collection screens but should not roll in active runs.
-

16. Content Status System

Purpose

The game may show planned content before it is fully implemented. This helps development planning and lets menus represent the intended final game.

Status Types

Implemented:

- Fully usable in current build.

MVP:

- Required for first playable version.

WIP:

- Visible in menus or data registries but not active in gameplay.

Future:

- Planned but not necessarily visible.

Disabled:

- Temporarily removed from gameplay selection.

Locked:

- Implemented or planned, but not available until unlocked.

Data Flags

Each ship, weapon, upgrade, shop item, enemy, attachment, and screen should eventually support status flags.

Example fields:

```
id
name
description
```

```
status
mvp
implemented
visible
enabled
tags
```

Active Gameplay Rule

WIP content should not appear in active gameplay random pools unless deliberately enabled for testing.

17. Visual Direction Placeholder

This section will be expanded after dedicated look-and-feel discussion.

Current direction:

- Simple geometric ships
- Color-coded enemies
- Polygon debris
- Bright projectile effects
- Clean sci-fi UI
- Dark starfield background
- Black holes with strong particle and distortion effects
- No complex art assets required for MVP

Future art section should define:

- Color palette
 - Ship silhouettes
 - Enemy shape language
 - Debris shape language
 - Projectile visual rules
 - Particle presets
 - UI layout and style
 - Animation language
 - Black hole visual identity
 - Menu style
-

18. Audio Placeholder

This section will be expanded after dedicated sound discussion.

Current direction:

- Arcade sci-fi sound effects
 - Clear weapon sounds
 - Strong impact feedback
 - Black hole ambience
 - Low-health warnings
 - Upgrade and shop confirmation sounds
 - Synth/electronic music direction, TBD
-

19. MVP Scope

MVP Goal

A playable browser prototype that proves the core loop is fun.

MVP Must Include

1. Modular TypeScript browser project
2. 9×9 viewport wrapping arena
3. Player movement with inertia
4. Two playable ships: Interceptor and Bulwark
5. Interceptor Pulse Cannon
6. Bulwark Ramming Shield
7. At least one moving black hole
8. Asteroids
9. Destroyed enemies creating hazard debris
10. At least three enemy types
11. Direct XP gain, no XP gems
12. XP bar and banked upgrades
13. Player-triggered upgrade overlay
14. 1-of-3 in-run weapon upgrades
15. Scrap pickups
16. End-run scrap-to-currency tally
17. Simple main-menu shop
18. Basic permanent shop upgrades
19. HUD
20. Results screen
21. WIP tags for visible non-MVP content

MVP Does Not Need

- Steam packaging
- Full meta-progression balance
- Bosses
- All ships

- All weapons
 - Full attachment system
 - Full sound design
 - Controller support
 - Achievements
 - Cloud saves
-

20. Post-MVP Features

Add only after the MVP is fun.

- More ships
 - More weapons
 - More attachments
 - More enemy types
 - Bosses
 - Elites
 - Minimap
 - Audio
 - Advanced shop upgrades
 - Controller support
 - Desktop wrapper
 - Steam integration
 - Achievements
 - Cloud saves
 - Steam playtest branch
-

AI DEVELOPMENT PLAN

21. AI Development Philosophy

Core Rule

AI should build the game in small, testable steps. It should not be asked to build the entire game at once.

ChatGPT Role

Use ChatGPT for:

- Design planning
- GDD maintenance
- System breakdowns

- Codex prompt writing
- Architecture review
- Balance design
- Debug explanation
- Steam page copy
- Task sequencing
- Art direction planning
- Sound direction planning

Codex Role

Use Codex for:

- Creating files
- Implementing systems
- Refactoring
- Fixing bugs
- Writing tests
- Building features from GitHub issues
- Creating pull requests

Human Role

The human developer must:

- Test every build
- Decide whether gameplay feels good
- Approve or reject AI changes
- Maintain scope discipline
- Keep the GDD updated
- Prevent feature creep

22. Recommended Repository Structure

```
src/  
  main.ts  
  scenes/  
    BootScene.ts  
    TitleScene.ts  
    GameScene.ts  
    ShipSelectScene.ts  
    ShopScene.ts  
    UpgradeScene.ts  
    ResultsScene.ts  
  core/
```

```
constants.ts
types.ts
math.ts
config.ts
status.ts
entities/
  Player.ts
  Enemy.ts
  Projectile.ts
  Pickup.ts
  Debris.ts
  Asteroid.ts
  BlackHole.ts
systems/
  InputSystem.ts
  MovementSystem.ts
  WeaponSystem.ts
  RamShieldSystem.ts
  EnemySpawnSystem.ts
  CollisionSystem.ts
  UpgradeSystem.ts
  LootSystem.ts
  BlackHolePhysicsSystem.ts
  ParticleSystem.ts
  ShopSystem.ts
  SaveSystem.ts
renderers/
  ShapeRenderer.ts
  BackgroundRenderer.ts
  BlackHoleRenderer.ts
  UIRenderer.ts
  ParticleRenderer.ts
data/
  ships.ts
  weapons.ts
  weaponUpgrades.ts
  shopUpgrades.ts
  enemies.ts
  attachments.ts
  statusRegistry.ts
```

23. Required Project Documents

Add these files to the repository:

```
README.md
STARVIVORS_GDD.md
ARCHITECTURE.md
CODING_RULES.md
VISUAL_STYLE_GUIDE.md
AUDIO_STYLE_GUIDE.md
CONTENT_STATUS_GUIDE.md
TASKS.md
BUGS.md
BALANCE_NOTES.md
PLAYTEST_NOTES.md
STEAM_RELEASE_CHECKLIST.md
```

Purpose of Each File

README.md: How to install, run, build, and test the project.

STARVIVORS_GDD.md: Current design source of truth.

ARCHITECTURE.md: How the codebase is organized.

CODING_RULES.md: Rules Codex must follow.

VISUAL_STYLE_GUIDE.md: Shape, color, particle, animation, and UI rules.

AUDIO_STYLE_GUIDE.md: Music and sound effect rules.

CONTENT_STATUS_GUIDE.md: Rules for Implemented, MVP, WIP, Future, Disabled, and Locked content.

TASKS.md: Current development roadmap.

BUGS.md: Known bugs and reproduction steps.

BALANCE_NOTES.md: Tuning notes from playtests.

PLAYTEST_NOTES.md: Human feel notes after each build.

STEAM_RELEASE_CHECKLIST.md: Packaging and release requirements.

24. Codex Workflow

Rule 1: One Task at a Time

Each Codex task should implement one system or one small feature.

Bad prompt:

Build the entire MVP.

Good prompt:

Implement player thrust, rotation, braking, and screen wrapping. Do not add weapons or enemies.

Rule 2: Always Include Acceptance Criteria

Every task should say how to verify completion.

Example:

- npm install works
- npm run dev starts the game
- Player rotates left/right
- Player thrusts forward
- Player wraps across arena edges
- No unrelated systems changed

Rule 3: Require Small Diffs

Codex should not rewrite large parts of the game unless explicitly asked.

Rule 4: Use GitHub Issues

Each feature should become a GitHub issue. Codex can work from the issue.

Rule 5: Review Before Merge

Never blindly merge AI-generated code. Test first.

25. Suggested Development Order

Phase 0 — Project Setup

- Create repo
- Add design docs
- Set up Phaser/Pixi + TypeScript + Vite
- Add basic scene
- Add placeholder player

Phase 1 — Movement Prototype

- Player ship movement
- Camera follow
- 9×9 arena
- Arena wrapping
- Debug HUD

Phase 2 — Combat Prototype

- Manual aiming
- Pulse Cannon
- Projectiles
- Asteroids
- Basic enemy

Phase 3 — Bulwark Prototype

- Bulwark ship
- Ramming Shield
- Shield charge/recharge
- Collision damage rules
- Ram damage rules

Phase 4 — Debris and Loot Prototype

- Enemy death debris
- Hazard debris collision
- Scrap pickups
- End-run resource tally

Phase 5 — Progression Prototype

- Direct XP gain
- XP bar
- Banked upgrades
- Upgrade button
- 1-of-3 weapon upgrade overlay
- Apply MVP weapon upgrades

Phase 6 — Black Hole Prototype

- Moving black hole
- Gravity pull
- Orbit/consume matter
- Visual danger rings
- Player danger behavior

Phase 7 — MVP Run Loop

- Timer
- Enemy escalation
- Health/shield
- Death
- Results screen
- Restart loop

Phase 8 — Menu and Shop

- Main menu
- Ship select
- Simple shop
- Permanent shop upgrades
- Save data

Phase 9 — Content Status/WIP Presentation

- Show WIP ships/weapons/upgrades visually
- Prevent WIP content from entering active gameplay
- Add status tags to data registry

Phase 10 — Content Expansion

- More weapons
- Attachments
- More ships
- More enemies
- Better particles
- Balance pass

Phase 11 — Desktop Build

- Add Electron/Tauri wrapper
- Package Windows build
- Test performance
- Add settings
- Add controller support

Phase 12 — Steam Preparation

- Store assets
- Trailer
- Steam build upload
- Achievements/cloud saves if desired
- Playtest branch
- Release checklist

26. First Codex Prompt

Create the initial STARVIVORS project scaffold.

Read STARVIVORS_GDD.md, ARCHITECTURE.md, CODING_RULES.md, and CONTENT_STATUS_GUIDE.md before coding.

Tech stack:

- TypeScript
- Vite
- Phaser 3 unless the repository specifies otherwise

Requirements:

- Create a working browser game project.
- Add a BootScene and GameScene.
- Render a simple triangle player ship in the center of the screen.
- Add a dark starfield background using simple generated shapes.
- Add a basic debug text showing FPS, player position, and arena size.
- Define the arena as 9×9 of the viewport.
- No enemies, weapons, black holes, upgrades, menus, or shop yet.

Acceptance criteria:

- npm install works.
- npm run dev starts the project.
- Browser shows the player triangle on a starfield.
- Debug text shows player position and 9×9 arena size.
- Code is organized according to ARCHITECTURE.md.
- No large unrelated systems are added.

COMPANION BREAKDOWN DOCUMENTS

These are intended to become separate repo documents later. They are kept here for now so the design remains centralized while the game direction is still being shaped.

Companion Doc A — Core Loop Breakdown

Purpose

Defines what the player does moment to moment, minute to minute, and run to run.

Moment-to-Moment Loop

- Thrust and drift through space
- Aim or position based on ship type
- Destroy enemies
- Avoid collision hazards
- Avoid black hole pull
- Destroy or evade debris
- Collect scrap pickups
- Watch XP bar fill
- Decide when to spend banked upgrades

Minute-to-Minute Loop

- Enemy density increases
- Black holes drift into and out of danger range
- Debris fields accumulate
- Player upgrades weapons
- Player chooses whether to fight, kite, clean debris, or reposition

Run-to-Run Loop

- Complete or fail run
- View results
- Convert collected scrap into credits
- Buy shop upgrades
- Unlock or preview ships/weapons
- Start a new run stronger or with a different ship

Companion Doc B — Arena and Movement Breakdown

Purpose

Defines the physical play space and player control model.

Arena

- 9×9 viewport world
- Wrapping boundaries
- Camera follows player
- Open-space feel
- Large enough for kiting, recovery, and black hole traversal

Movement Goals

- Floaty but controllable
- Skill-based momentum

- Clear acceleration and braking
- Enough drift to feel like space
- Enough control to avoid frustration

Early Tuning Variables

- Max speed
 - Thrust acceleration
 - Brake strength
 - Turn rate
 - Passive drag/friction
 - Dash strength
 - Dash cooldown
 - Collision bounce
 - Black hole pull resistance
-

Companion Doc C — Black Hole Breakdown

Purpose

Defines the signature gravity hazard.

Core Features

- Moves slowly across arena
- Pulls physical entities
- Curves projectiles
- Consumes matter at center
- Creates danger zones
- Can be exploited against enemies

Required Entity Interactions

- Player: pulled, damaged/consumed near center
- Enemies: pulled, can be destroyed
- Debris: pulled, orbits, consumed
- Projectiles: curved, consumed if too close
- Pickups: lightly pulled or fully pulled depending on tuning

MVP Tuning Needs

- Pull strength
- Pull falloff
- Consume radius
- Danger radius
- Movement speed
- Warning indicators

- Escape feasibility
-

Companion Doc D — Ships Breakdown

Purpose

Defines ship identity, starting weapons, and playstyles.

MVP Ships

Interceptor:

- Ranged
- Pulse Cannon
- Fragile but flexible
- Good first-player baseline

Bulwark:

- Melee/impact
- Ramming Shield
- Durable when shielded
- High-risk, high-reward collision play

WIP Ship Presentation

All planned ships can appear visually in ship select with WIP tags. Only implemented ships are selectable.

Future Ship Ideas

- Prospector: resource-focused mining ship
 - Storm: chain lightning ship
 - Phantom: dash/phase ship
 - Hive: drone carrier
 - Singularity: black-hole-resistant experimental ship
-

Companion Doc E — Weapons and Upgrades Breakdown

Purpose

Defines active weapons, weapon upgrades, and how in-run progression works.

Weapon Rules

- 2 active weapon slots

- Slot 1 is ship-specific
- Slot 2 is acquired during the run
- MVP focuses on starting weapons first

MVP Weapons

Pulse Cannon:

- Slow baseline ranged weapon
- Starts at roughly 1.25 second cooldown
- Upgrades into stronger fire rate, damage, projectile count, pierce, and size

Ramming Shield:

- Movement/collision-based weapon
- Uses shield charge
- Requires disengage/recharge rhythm
- Upgrades into better melee damage, recharge, shield capacity, and impact radius

Upgrade Rules

- XP creates banked upgrades
- Banked upgrades are infinite
- Player opens upgrade screen manually
- Gameplay pauses during selection
- Only wired upgrades appear in active run choices
- WIP upgrades may be visible elsewhere

Companion Doc F — Enemies, Debris, and Pickups Breakdown

Purpose

Defines hostile entities, cleanup hazards, and resource drops.

Enemy Goals

- Pressure movement
- Create targets
- Break into debris
- Interact with black holes
- Support both ranged and melee ship playstyles

Hazard Debris Goals

- Create aftermath from combat
- Add cleanup decisions
- Give extra XP opportunities

- Threaten careless movement
- Build screen pressure without always adding active enemies

Pickup Goals

- Reward combat
- Feed end-run economy
- Create risk when pickups drift near black holes

MVP Enemy Needs

At least three enemy types:

- Basic chaser
 - Shooter or ranged ship
 - Tank or heavy ship
-

Companion Doc G — Shop and Meta-Progression Breakdown

Purpose

Defines permanent progression outside runs.

Shop Style

Simple Vampire Survivors-style upgrade shop. No sphere grid.

MVP Shop Categories

Ship/survival:

- Max Hull
- Shield Capacity
- Thrust
- Pickup Range

Weapon:

- Weapon Damage
- Attack Speed
- Melee Damage
- Ram Shield Recharge

Economy:

- Scrap Value

Rules

- Shop upgrades are purchased with credits
 - Credits come from run resource tally
 - Shop upgrades persist across runs
 - Unlocks can be added later
-

Companion Doc H — UI and Screens Breakdown

Purpose

Defines all menus and HUD elements.

MVP Screens

- Main Menu
- Ship Select
- Shop
- Game HUD
- Upgrade Overlay
- Pause Menu
- Results Screen

HUD Priorities

- Readability over decoration
- XP bar at top
- Upgrade bank count visible
- Hull/shield visible
- Scrap count visible
- Weapon status visible
- Black hole danger readable

WIP Rules

- WIP content should be visible only where useful
 - WIP content must be clearly tagged
 - WIP content should not look broken or accidentally selectable
-

Companion Doc I — Content Status Breakdown

Purpose

Defines how unfinished content appears in the game and codebase.

Status Values

- Implemented
- MVP
- WIP
- Future
- Disabled
- Locked

Active Run Rule

Only enabled, implemented content can enter active random gameplay pools.

Menu Rule

WIP content can appear in menus if clearly tagged.

Codex Rule

Codex must not wire WIP content into gameplay unless the task explicitly says to implement it.

Companion Doc J — AI Production Breakdown

Purpose

Defines how to use ChatGPT and Codex safely.

Recommended Workflow

1. Discuss and define design in ChatGPT.
2. Update GDD and companion docs.
3. Convert one small feature into a Codex-ready task.
4. Codex implements in a branch or PR.
5. Human tests.
6. Bugs and balance notes are written down.
7. Merge only after review.
8. Repeat.

Best Codex Task Format

```
Task:  
Context:  
Files to read:  
Requirements:  
Do not change:
```

Acceptance criteria:
Testing instructions:

Bad AI Pattern

Asking for a whole game or large system cluster at once.

Good AI Pattern

Asking for one isolated, testable system at a time.

OPEN DESIGN QUESTIONS

These should be resolved in future design sessions.

1. Phaser 3 or PixiJS?
2. Exact starting stats for Interceptor and Bulwark?
3. Exact Pulse Cannon projectile behavior and visual scale?
4. Exact Ramming Shield collision formula?
5. Should black holes consume pickups at full strength or reduced strength?
6. How should the second weapon be acquired?
7. Which three enemy types are MVP?
8. What does the shop look like visually?
9. What art palette defines STARVIVORS?
10. What is the sound/music identity?
11. How much WIP content should be visible in early builds?
12. Should bosses appear in MVP or first post-MVP update?

CURRENT AGREED DIRECTION

- Modular TypeScript project
- Browser prototype first
- Steam wrapper later
- Simple shapes and particles
- 9×9 viewport wrapping arena
- Asteroids-inspired movement
- Moving black holes
- Direct XP gain, no XP gems
- Infinite banked upgrades
- Player-triggered upgrade screen
- In-run upgrades are weapon-focused
- Post-run shop handles ship/survival/weapon meta upgrades
- Simple shop, no sphere grid

- MVP has two ships: Interceptor and Bulwark
- Interceptor starts with slow Pulse Cannon
- Bulwark starts with Ramming Shield
- Destroyed enemies create hazardous debris
- Scrap/minerals become end-run currency
- WIP content may appear visually but is not wired into active gameplay unless implemented